

Context Shaping for Reduced-Round SHA-256 Compression-Function Preimages: Nineteen Rounds in Milliseconds, and Computed Preimages at Twenty Rounds

Kameldip Singh Basra

Independent Researcher

kameldipbasra@gmail.com

ORCID: 0009-0001-0977-4614

September 2026

Abstract

The table-based preimage attack of [1] on the 19-round SHA-256 compression function chooses seven internal state words, the *context*, at random, and then pays twice: a fixed-point iteration that converges for about one C_0 survivor in 2^{16} , and an exact 32-bit consistency check on a provisional value of the message word W_9 . We show that both costs vanish on a large subfamily of contexts. Choosing the context so that $a_4 = a_5$ and two derived words satisfy $e_8 = e_9 = 0xFFFFFFFF$ makes the three schedule constraints solve triangularly in three table lookups per attempt, with no iteration, and collapses the consistency residual to $\text{Maj}(a_4, a_3, a_2) - a_3 \bmod 2^{32}$, whose vanishing is a bitwise condition of probability exactly $(3/4)^{32}$ rather than 2^{-32} . The measured cost of a 19-round preimage falls from about $2^{38.3}$ swept values of a_0 to about $2^{15.25}$, within one per cent of the derived $1/(c^3(3/4)^{32}) = 39,124$: about 12 milliseconds of one CPU core after a reusable one-time table precomputation. We exhibit 13,840 verified preimages produced in 152 seconds, and 108 preimages of the all-ones digest that Zaikin [12] inverted at 19 rounds in 18 h 33 min on 192 cores.

At 20 rounds the fourth schedule constraint remains an exact 32-bit filter. We measure its residual to be uniform to 24 bits and find, by a characteristic-function analysis with a validated positive control, no bias that a table representative policy or a context choice could exploit: the first-harmonic contribution is below 10^{-3} bits at our sample size. The cost is therefore $2^{45.4}$ swept a_0 with three-root tables. With a fused GPU kernel this is about 15 hours on one NVIDIA A100. We report two computed and verified 20-round preimages, one for the all-ones digest, found after 16 h on one of two A100s searching it (about \$26 on that GPU, \$52 for the pair, against an expected 15 h and \$24 per preimage). To our knowledge, and after the search described in Section 8 (as of September 2026, covering the IACR ePrint archive, dblp, OpenAlex, arXiv and the SAT, CP and AI venues), no computed preimage of the SHA-256 compression function beyond 19 rounds has been published. We also give evidence that, for this construction, 21 rounds costs the full additional 2^{64} : the fourth and fifth constraints are measured to be independent, and the dependency that blocks a further collapse passes through Σ_0 of the unknown itself, which no context word can saturate. Neither lever is new on its own; the contribution is their combination and the bitwise collapse. Nothing here affects full SHA-256.

1 Introduction

A *preimage* of the SHA-256 compression function reduced to R rounds is a 512-bit block $W_0, \dots, W_{15}$ such that the R -round compression of the standard initial value with that block,

plus the feed-forward, equals a given 256-bit digest. We keep the feed-forward and drop the padding constraint, the model used by Zaikin [12] and by our earlier paper [1]; a preimage in this model is strictly stronger than a pseudo-preimage, in which the initial value is free.

Two lines of work address this problem. The meet-in-the-middle line reaches high step counts at costs just below brute force: 43 steps at $2^{254.9}$ [5], 45 at $2^{255.5}$ [6], and, since February 2026, 47 at $2^{255.6}$ [7]. None of those attacks has been executed in full (the largest computed fragment is discussed in Section 8), and the cheapest figure anywhere in that line is 2^{240} at 24 steps [4]. The practical line, which actually computes preimages, has moved slowly: 16 rounds by SAT solving in 2012 [8], 17 and 18 rounds for the all-zero digest in 2025 [9], and 19 rounds in January 2026, when Zaikin [12] inverted the all-ones digest in 18 h 33 min on 192 CPU cores and wrote that “it is likely that additional effective algorithmic techniques are required to invert . . . 20-round SHA-256 compression functions” ([12], §8). Our earlier paper [1] reached 19 rounds by a different, algebraic route, at about two GPU-hours per preimage of an arbitrary digest, and a companion note [2] put the cost of the same family at 20 rounds at 2^{64} per context-equivalent, 2^{32} times the 19-round per-context cost, about 2^{71} in total at the 19-round rate measured there.

This paper corrects both of our own figures downward by large factors, and reports two computed and verified preimages of the 20-round SHA-256 compression function. To our knowledge, and after the search described in Section 8 (as of September 2026), no computed preimage of that function beyond 19 rounds had previously been published.

What the base attack does. The attack of [1] fixes the last eight state words from the digest by a backward chain, chooses the seven state words $a_4, \dots, a_{10}$ at random (the *context*), sweeps a_0 over all 2^{32} values, and recovers a_1, a_2, a_3 from the three message-schedule constraints on W_{16}, W_{17}, W_{18} using a 16 GB table that inverts $u \mapsto \sigma_0(u) - u$. Two things make it expensive. The constraints on W_{17} and W_{18} are coupled through the round function, so a_2 and a_3 are found by a fixed-point iteration that converges for roughly one C_0 survivor in 2^{16} ($1.3\text{--}1.9 \times 10^{-5}$ per survivor in [1]). And the constraint on W_{16} needs W_9 , which depends on the still-unknown a_2, a_3 ; the attack freezes them at zero, solves, and afterwards tests that the true W_9 equals the provisional one, a 32-bit condition that holds for about one converged candidate in 2^{22} .

What this paper does. We choose the context instead of drawing it at random. Three conditions, $a_4 = a_5$ together with $e_8 = e_9 = 0\text{x}\text{FFFFFFFF}$ (the last two each imposed by solving one round equation for one context word), have two consequences (Section 3). The coupling that forced the iteration disappears: the constraint on W_{17} loses its dependence on a_3 , so a_1, a_2, a_3 are obtained by three table lookups. And the consistency residual collapses to $\text{Maj}(a_4, a_3, a_2) - a_3$, a difference of two bitwise functions that is borrow-free, so its vanishing is the per-bit condition $\text{Maj}(a_4, a_3, a_2) = a_3$, satisfied by exactly 3^{32} of the 2^{64} pairs.

Results.

- At 19 rounds the cost per preimage is about $2^{15.25}$ swept a_0 , pooled over 1.14×10^9 swept values and 29,398 verified preimages in three runs, against $2^{38.3}$ for the random-context attack: a factor of about 2^{23} , with an interval of $2^{21.5}$ to $2^{25.3}$ inherited from the baseline’s own uncertainty (Section 5). The independently derived value is $1/(0.633673^3 \cdot (3/4)^{32}) = 39,124$; the three-run pooled value is 38,878 (1,142,947,840 swept a_0 over 29,398 preimages), and the largest single independent replication (Table 1, 24 contexts) alone gives 39,123. One CPU core produces a preimage of an arbitrary digest in about 12 ms. The all-ones digest of [12] and the all-zero benchmark of [9] each yield about a hundred preimages per second.
- At 20 rounds the fourth schedule constraint is an exact 32-bit filter in which we find no bias that a table policy or a context choice could exploit (Section 6). With three-root

tables and a fused GPU kernel the cost is $2^{45.4}$ swept a_0 , about 15 hours on one A100. We computed two 20-round preimages: one for a digest generated by the solver, after 2.45 h on one A100, and one for the all-ones digest, after 16.2 h on one of two A100s searching that digest (about \$26 on that GPU, \$52 for the pair). To our knowledge, and after the search described in Section 8 (as of September 2026), no computed preimage of the SHA-256 compression function beyond 19 rounds had previously been published.

- Twenty-one rounds costs this construction the full additional 2^{64} (Section 7). On 214,583 freshly generated 21-round candidates the fourth and fifth constraint residuals are uniform and independent, and a symbolic search over 1,485 context families finds no further collapse. The reason is structural: the technique clears a dependency only when it passes through Maj or Ch, which a context word can saturate, and the dependency that survives passes through $\Sigma_0(a_4)$, where there is no context word to set.

What is and is not new. The two levers are known. Saturating the choice function, $\text{Ch}(\text{0xFFFFFFFF}, y, z) = y$, is the “absorption property” used for message stealing by Aoki et al. [5]; the majority degeneracy $\text{Maj}(x, x, y) = x$ is standard and appears in Guo et al. [7] as an auxiliary condition that absorbs propagation through Maj (their Proposition 11 and Remark 3). Our own companion note [2] applies both levers to a different constraint and dismisses the result because one of its conditions tied a context word to an unknown, which would force the constants to be rebuilt per candidate. What is new here is the specific pairing $a_4 = a_5$, which ties two *context* words together and costs nothing per candidate, its application to the constraint that carries the iteration, and the observation that the same choice turns a 32-bit modular consistency check into a bitwise one. We have found no precedent for that last effect. The barrier argument of [2] was correct about where the wall stands and about its height relative to the 19-round cost, a factor 2^{32} ; what it did not anticipate is that the 19-round cost itself could fall by 2^{23} . Section 9 reconciles the two.

Scope. Nineteen and twenty of SHA-256’s 64 rounds. Nothing in this paper bears on full SHA-256; on full SHA-224, which shares the compression function but uses a different initial value and truncates its output; on SHA-512, for which the table would need 2^{64} entries; or on any deployed system.

2 Preliminaries

Notation. All words are 32-bit and arithmetic is modulo 2^{32} unless stated. $\Sigma_0, \Sigma_1, \sigma_0, \sigma_1$ are the SHA-256 functions of FIPS 180-4 [3], $\text{Ch}(e, f, g) = (e \wedge f) \oplus (\neg e \wedge g)$ and $\text{Maj}(a, b, c) = (a \wedge b) \oplus (a \wedge c) \oplus (b \wedge c)$. Following [1], the state after round r is written a_r and e_r , with $a_{-1}, \dots, a_{-4}$ and $e_{-1}, \dots, e_{-4}$ the eight words of the initial value. Round r consumes W_r :

$$\begin{aligned} T_1^{(r)} &= e_{r-4} + \Sigma_1(e_{r-1}) + \text{Ch}(e_{r-1}, e_{r-2}, e_{r-3}) + K_r + W_r, \\ T_2^{(r)} &= \Sigma_0(a_{r-1}) + \text{Maj}(a_{r-1}, a_{r-2}, a_{r-3}), \end{aligned}$$

$a_r = T_1^{(r)} + T_2^{(r)}$ and $e_r = a_{r-4} + T_1^{(r)}$. Hence $e_r = a_{r-4} + a_r - T_2^{(r)}$ is determined by the a -words alone, and every message word is recovered from the state:

$$W_r = a_r - T_2^{(r)} - e_{r-4} - \Sigma_1(e_{r-1}) - \text{Ch}(e_{r-1}, e_{r-2}, e_{r-3}) - K_r. \quad (1)$$

The digest after R rounds is the initial value plus $(a_{R-1}, a_{R-2}, a_{R-3}, a_{R-4}, e_{R-1}, \dots, e_{R-4})$.

The base attack. Given the digest, the backward chain of [1] recovers $a_{R-8}, \dots, a_{R-1}$. At $R = 19$ the free words are $a_0, \dots, a_{10}$; the attack fixes $a_4, \dots, a_{10}$ (the context), sweeps a_0 , and must solve for a_1, a_2, a_3 against the schedule constraints

$$C_j : \quad W_{16+j} = \sigma_1(W_{14+j}) + W_{9+j} + \sigma_0(W_{1+j}) + W_j, \quad j = 0, 1, 2, \quad (2)$$

in which every W is expressed through (1). Because a_{j+1} enters W_{j+1} with coefficient +1 and W_{9+j} with coefficient -1, C_j can be written as $\sigma_0(W_{j+1}) - W_{j+1} = (\text{terms free of } a_{j+1})$, and a global table inverting $u \mapsto \sigma_0(u) - u$ yields W_{j+1} , hence a_{j+1} , in one lookup. The map $u \mapsto \sigma_0(u) - u$ is not a bijection: its image has exactly 2,721,603,628 of the 2^{32} values, a fraction 0.633673, and the published table stores one root per image value, the largest.

The difficulty is that the right-hand side of C_1 depends on a_3 and that of C_2 on a_2 , so the attack iterates $C_1 \rightarrow a_2$, $C_2 \rightarrow a_3$ to a fixed point; and the right-hand side of C_0 contains W_9 , which depends on a_2, a_3 through

$$W_9 = \underbrace{T_1^{(9)} - K_9}_{\text{context}} - e_5 - \Sigma_1(e_8) - \text{Ch}(e_8, e_7, e_6), \quad e_5 = a_1 + a_5 - \Sigma_0(a_4) - \text{Maj}(a_4, a_3, a_2), \quad (3)$$

with $e_6 = a_2 + T_1^{(6)}$ and $e_7 = a_3 + T_1^{(7)}$. The attack evaluates W_9 at $a_2 = a_3 = 0$, a provisional value $\hat{W}_9$, solves C_0 with it, and finally tests

$$\varepsilon = W_9^{\text{conv}}(a_2, a_3) - W_9^{\text{conv}}(0, 0) \stackrel{?}{=} 0, \quad (4)$$

where W_9^{conv} is the a_1 -free part of (3). The measured rates in [1] are about 1.6×10^{-5} fixed points per C_0 survivor and one success of (4) per 2^{22} fixed points, giving about one preimage per 80 attempts of 2^{32} swept a_0 each, i.e. $2^{38.3}$ swept a_0 per preimage.¹

3 The Submask Context Family

Definition 1 (The family). *A context is in the submask family if*

$$a_4 = a_5 = v \quad \text{for some } v, \quad e_8 = 0\text{x}\text{FFFFFFFF}, \quad e_9 = 0\text{x}\text{FFFFFFFF}.$$

Since $e_8 = a_4 + a_8 - T_2^{(8)}$ and $e_9 = a_5 + a_9 - T_2^{(9)}$, the last two conditions are met by choosing a_6, a_7 freely and then setting $a_8 = -1 - a_4 + \Sigma_0(a_7) + \text{Maj}(a_7, a_6, a_5)$ and $a_9 = -1 - a_5 + \Sigma_0(a_8) + \text{Maj}(a_8, a_7, a_6)$. The words v, a_6, a_7, a_{10} (and a_{11} at $R = 20$) remain free, so the family contains 2^{128} contexts at 19 rounds and 2^{160} at 20.

Proposition 1 (The iteration disappears). *In the submask family the right-hand side of C_1 does not depend on a_3 . Consequently, for each swept a_0 , a_1, a_2 and a_3 are obtained by three successive table lookups, $C_0 \rightarrow a_1$, $C_1 \rightarrow a_2$, $C_2 \rightarrow a_3$, and every triple so obtained satisfies C_0 (with the provisional $\hat{W}_9$), C_1 and C_2 exactly.*

Proof. The only route by which a_3 reaches C_1 is through W_{10} in (2), and by (1) $W_{10} = \text{const} - e_6 - \text{Ch}(e_9, e_8, e_7)$ with $e_6 = a_2 + a_6 - \Sigma_0(a_5) - \text{Maj}(a_5, a_4, a_3)$ and $e_7 = a_3 + T_1^{(7)}$. The a_3 -dependence is therefore

$$D(a_3) = \text{Maj}(a_5, a_4, a_3) - \text{Ch}(e_9, e_8, a_3 + T_1^{(7)}).$$

With $a_5 = a_4$, $\text{Maj}(a_4, a_4, a_3) = a_4$; with $e_9 = 0\text{x}\text{FFFFFFFF}$, $\text{Ch}(e_9, e_8, \cdot) = e_8$. Both terms are constants, so D is constant. C_2 's right-hand side depends on a_2 (and on context) but its own unknown a_3 cancels as in the base attack. The three constraints are therefore triangular in (a_1, a_2, a_3) and each lookup either returns the unique stored root or misses. $\square$

¹The pooled campaign estimate of [1], $\hat{\mu} = 3/240$ per attempt, mixes a screened arm with a control arm that produced no events, and carries a 95% interval of $[2.6 \times 10^{-3}, 3.7 \times 10^{-2}]$. Any point in that interval is what this paper improves on by about 2^{23} ; the distinction does not affect the comparison.

Neither condition suffices alone: an isolation test on three targets gave 0–16 triangular solutions per 2^{22} swept a_0 for random contexts, 3–391 with $a_5 = a_4$ only, 2–236 with $e_9 = 0\text{x}\text{FFFFFFFF}$ only, and about 1.07×10^6 with both, i.e. 0.402 per C_0 survivor, which is 0.634^2 , the probability that the two further lookups both hit.

Proposition 2 (The consistency check collapses). *In any context with $a_4 = v$ and $e_8 = 0\text{x}\text{FFFFFFFF}$ (the condition $a_5 = a_4$ is not needed for this part),*

$$\varepsilon = \text{Maj}(v, a_3, a_2) - a_3 \pmod{2^{32}}. \quad (5)$$

Moreover the subtraction is borrow-free: with $P = a_3 \wedge \neg a_2 \wedge \neg v$ and $Q = a_2 \wedge \neg a_3 \wedge v$, one has $\text{Maj}(v, a_3, a_2) = a_3 - P + Q$ with $P \subseteq a_3$ and $Q \cap a_3 = \emptyset$. Hence $\varepsilon = 0$ if and only if $\text{Maj}(v, a_3, a_2) = a_3$ as bit strings, equivalently

$$(a_2 \oplus a_3) \wedge (a_3 \oplus v) = 0, \quad (6)$$

which holds for exactly 3^{32} of the 2^{64} pairs (a_2, a_3) , i.e. with probability $(3/4)^{32} = 1.004524 \times 10^{-4}$ for uniform independent a_2, a_3 , for every v .

Proof. With $e_8 = 0\text{x}\text{FFFFFFFF}$, $\text{Ch}(e_8, e_7, e_6) = e_7 = a_3 + T_1^{(7)}$ in (3), so the a_1 -free part of W_9 is $W_9^{\text{conv}} = \text{const} + \Sigma_0(v) - a_5 + \text{Maj}(v, a_3, a_2) - a_3 - T_1^{(7)}$. Evaluating at $a_2 = a_3 = 0$ gives the same expression with $\text{Maj}(v, 0, 0) = 0$ and $a_3 = 0$, and subtracting yields (5). For the decomposition, compare bits: where $v_i = 0$, Maj is $a_{3,i} \wedge a_{2,i}$, which differs from $a_{3,i}$ exactly when $a_{3,i} = 1, a_{2,i} = 0$, the bits of P ; where $v_i = 1$, Maj is $a_{3,i} \vee a_{2,i}$, which differs from $a_{3,i}$ exactly when $a_{3,i} = 0, a_{2,i} = 1$, the bits of Q . Since $P \subseteq a_3$, $a_3 - P$ clears bits without borrows, and since Q is disjoint from $a_3 - P$, adding Q sets bits without carries. Thus $\varepsilon = Q - P$ with P, Q disjoint, which is zero iff $P = Q = 0$, i.e. iff each bit position avoids the two forbidden patterns; the surviving pairs number 3^{32} . $\square$

At $v = 0$ condition (6) reads $a_3 \wedge \neg a_2 = 0$: a_3 is a submask of a_2 , which names the family. The exactness of Proposition 2 was checked on 2000 random contexts and pairs, and its probability on 2^{22} random pairs at $v \in \{0, 0\text{x}\text{FFFFFFFF}, 0\text{x}6\text{A}09\text{E}667\}$, by `code/submask_family.py` in the repository, which needs no table.

Corollary 1 (Yield). *Model the three lookups as returning words that are uniform and independent of v and of one another; the count 3^{32} is exact for uniform independent pairs, and Section 5 tests the model. Under it, with a single-root table of image fraction $c = 0.633673$, each swept a_0 yields $c^3 = 0.2545$ triangular solutions, of which a fraction $(3/4)^{32}$ satisfy (4). Every such triple is a 19-round preimage. The expected number of swept a_0 per 19-round preimage is therefore*

$$\frac{1}{c^3 (3/4)^{32}} = 39,124 = 2^{15.26}.$$

The last claim of the corollary, that the collapsed condition is *sufficient*, was verified directly: on 64,000 triangular solutions recomputed from their message words, C_1 and C_2 held for all, C_0 held exactly for those with $\varepsilon = 0$, and those were preimages.

Why it works, in one sentence. The base attack’s iteration converges for about one C_0 survivor in 2^{16} rather than at the 2^{-29} of a random map on a 32-bit state [2], because the feedback into C_1 passes through a bitwise select rather than through the whole round function; the family removes that feedback altogether, and the same saturation that removes it leaves the consistency residual as a difference of two bitwise functions.

4 Implementation

Tables. The published table stores one root per image value, the largest. A random target has a Poisson-like number of roots (the exact image fraction 0.633673 is slightly above $1 - 1/e = 0.632121$, and the multiplicity distribution is slightly under-dispersed), so keeping only one discards a factor $1/c = 1.578$ of the available continuations at each lookup. Building second- and third-root tables (first free slot, u distinct from the roots already stored) raises the per-lookup coverage to 0.8979 and 0.9774, and the triangular solutions per swept a_0 to 0.7240 and 0.9338; measured values are 0.725 (`logs/smoke19_2roots.log`) and 0.9338 (`status.json` of each pod, solutions divided by swept a_0). Each table is 16 GB; three fit an 80 GB GPU. The gain in wall-clock is smaller than the gain in solutions, since the extra branches cost lookups: about $1.4\times$ on a CPU for two roots (2.80×10^5 against 2.06×10^5 swept a_0 per second, `logs/smoke19_1root.log` and `logs/smoke19_2roots.log`); the GPU ratio was not measured separately. One index in 2^{32} is lost to the sentinel (the value `0x20000000` has the single root `0xFFFFFFFF`, which encodes the “miss” sentinel); it is documented and irrelevant to any rate.

Kernel. A first GPU implementation in eager PyTorch reached 4.47×10^8 swept a_0 per second on an H100 and 2.54×10^8 on an A100, bound not by the table gathers but by streaming 64-bit intermediates through memory. A fused Triton kernel that keeps the whole chain in 32-bit registers, gathers only from the three tables, and records only the winning a_0 for host-side reconstruction reaches $9.1\text{--}9.3 \times 10^8$ per second kernel-timed on an A100 (`FLEET.txt`), roughly $3.6\times$ the eager kernel on the same GPU, and sustained $8.6\text{--}9.0 \times 10^8$ in production including host overhead (`status.json` of each Triton pod, swept a_0 divided by elapsed time), i.e. 4.7–5.0 s per context of 2^{32} swept a_0 . Averaged over each pod it produces 402,880–402,910 collapsed-condition candidates per context (`status.json`, candidates divided by contexts), against $0.93384 \cdot (3/4)^{32} \cdot 2^{32} = 402,897$ predicted from the exact three-table yield 0.977443^3 . Two implementation traps are worth recording: Triton’s right shift is arithmetic, so every rotation must mask, and a 32-bit table index sign-extends into an illegal address unless masked to 64 bits. The kernel was validated against the reference implementation on planted preimages before use (Section 6.4).

Reference implementation and verification. Every reported preimage is rebuilt from its recovered state into sixteen message words and re-hashed by an implementation independent of the solver before it is reported, and again by the standalone verifier `code/verify_r19.py` (standard library only, no shared code with the attack), which prints `result: OK`.

5 Nineteen Rounds

Table 1 summarises the 19-round measurements. The 64-target run took 151.7 s of wall time in one unpinned numpy process (3.5×10^6 swept a_0 per wall-second); pinned to a single core the rate is 3.4×10^6 swept a_0 per core-second (`reviewers/reports.json`, `taskset` with `OMP_NUM_THREADS=1`), so a preimage of an arbitrary digest costs about 12 ms of one core. The measured collapse rate, 1.0132×10^{-4} per solution on the 64-target run and $1.0122 \times 10^{-4} \pm 1.1 \times 10^{-6}$ on a second, larger reviewer replication (3.36×10^8 swept a_0), agrees with $(3/4)^{32} = 1.0045 \times 10^{-4}$; an absolute yield audit on a separate corpus (4.70×10^9 swept a_0 over 100 contexts at 19 and 20 rounds, 119,948 collapsed-condition candidates, every 19-round one a verified preimage), observed divided by the prediction of Corollary 1 with the measured table fill 0.63375, gives 0.9986 ± 0.0029 , so any hidden gain or loss lies within a hundredth of a bit. The rate does not depend on the target: the all-ones and all-zero benchmark digests yield at the same rate as digests of random messages.

Table 1: Nineteen-round preimages from the submask family, single-root table, CPU (numpy). “Solutions” are triangular solutions; “ $\varepsilon = 0$ ” are verified preimages.

Run	swept a_0	solutions	per survivor	$\varepsilon = 0$	a_0 /preimage
4 random targets	16,777,216	4,270,445	0.4017	395	42,474
64 random targets	536,870,912	136,596,846	0.4015	13,840	38,791
pooled, independent replication ^a	270,532,608	—	—	6,915	39,123
second independent replication ^a	335,544,320	85,389,673	—	8,643	38,823
all-ones digest [12]	4,194,304	1,068,641	0.4020	108	38,836
all-zero digest [9]	4,194,304	1,068,359	0.4019	115	36,472

^a Twenty-four contexts, twenty-four targets and twelve values of v , run from a fresh implementation in an independent AI-agent review pass (see the disclosure); 6,915 of 6,915 collapsed-condition candidates verified as preimages; the second replication, 8,643 of 8,643. Derived value: 39,124. The four-target row is the run reproduced bit for bit in `reviewers/reports.json`; the other rows are in `logs/r19_big.log` and `logs/r19_fixed_targets.log`.

Table 2: Nineteen-round preimages of the SHA-256 compression function, standard IV, all 256 bits fixed. Costs are per preimage.

	Zaikin [12]	[1]	this work
Targets	one fixed (all-ones)	arbitrary	arbitrary
Method	tuned Kissat inside Cube-and-Conquer	random context, fixed-point iteration	submask family, three lookups
Cost	18 h 33 min on 192 cores ^a	$2^{38.3}$ swept a_0 ; about 2 GPU-hours	$2^{15.26}$ swept a_0 ; about 12 ms of one CPU core
Preimages verified	1	7	more than 20,000 ^b

^a About 3,560 core-hours; obtained from an intermediate problem whose solutions are 19-round preimages with probability 1/8, so the expected cost of that route is about eight times the reported figure. Direct attempts on the 19-round instance found nothing in seven days on 192 cores (his account). ^b Counts are in the logs; verified blocks with their targets are printed in `logs/r19_scale.log` (33), `logs/r19_fixed_targets.log` (4), `logs/r19_variants.log` and `logs/r19_degenerate.log` (2 each) and the `logs/smoke19_*.log` files.

Comparison. Table 2 compares like with like. Two cautions apply. The new cost is a *marginal* cost: both our attacks presuppose the same one-time 16 GB table, which takes about 215 core-seconds to build, so a cold-start cost for a single preimage is dominated by the table. And the baseline of [1] carries its own uncertainty (a 95% interval of 2.6×10^{-3} to 3.7×10^{-2} on the per-attempt rate), so the speedup is about 2^{23} , with an interval of $2^{21.5}$ to $2^{25.3}$. Against Zaikin [12] the marginal ratio in core-time is about 10^9 ; including the table build for a single preimage from cold it is about 6×10^4 . His 18 h 33 min figure is itself a single favourable draw: by his account the solution of the intermediate problem he solved is a 19-round preimage with probability 1/8.

6 Twenty Rounds

At $R = 20$ the backward chain fixes $a_{12}, \dots, a_{19}$, the context is $a_4, \dots, a_{11}$, and a fourth constraint C_3 on W_{19} appears. The submask family transplants unchanged: Propositions 1 and 2 involve only $a_4, \dots, a_9$ and W_9 , and the measured rates at 20 rounds are 0.4015 solutions per C_0 survivor on 40 targets of 2^{26} swept a_0 each (`logs/c3_uniformity.log`) and a collapse rate of 1.0049×10^{-4} per solution on 8.4×10^{10} swept a_0 (`logs/c3_scaling.log`). Everything therefore reduces to the residual of C_3 ,

$$c_3 = \sigma_0(W_4) + W_3 - \kappa_3, \quad \kappa_3 = W_{19} - \sigma_1(W_{17}) - W_{12}, \quad (7)$$

which must vanish, where W_3, W_4 depend on $a_0, \dots, a_4$ and κ_3 on the context and digest only.

6.1 The fourth constraint is a flat filter

We tested the distribution of c_3 on candidates that satisfy everything else.

- A chi-squared test on 68,414 candidates, on the low and the high 4, 6, 8, 10 and 12 bits: every $|z| \leq 2.1$ (largest: high 10 bits, $z = +2.10$).
- A scaling ladder on 8.4×10^{10} swept a_0 (2.14×10^6 candidates): the number with the low k bits of c_3 zero, against the uniform prediction, has ratio 1.01 at $k = 8$, 0.96 at 12, 1.04 at 16, 0.86 at 18 (7 against 8.2).
- The GPU campaign’s own counters (`gpu_run_snapshot/h100_status.json`, 2.96×10^9 candidates): nested thresholds are correlated, but they decompose into independent conditional halvings, each $\text{Binomial}(n_k, \frac{1}{2})$. Aggregated over steps 20 to 28 the H100 pod gives 2,654 successes of 5,370 (expected 2,685), $z = -0.85$; the worst of those eight steps (25 $\rightarrow$ 26, 78 to 28, one-sided binomial $p = 0.008$) is $p \approx 0.07$ after correcting for the eight steps examined, and the two A100 pods give $z = +1.47$ (754 of 1,452) and $z = +1.71$ (759 of 1,453) on the same steps (+1.01 and +1.35 when the step 19 $\rightarrow$ 20 is included). Including the step 19 $\rightarrow$ 20, on which the H100 alone is low, the pooled three-GPU aggregate is 8,285 of 16,712 (expected 8,356), $z = -1.10$.
- A dedicated test on the all-ones digest itself, against random digests as a matched control (`logs/ones_obstruction.log`; 3.95×10^6 candidates in each arm): no deficit that deepens with k appears in either arm: every cell of the all-ones arm from 8 bits on is within 1.3σ of expectation below 18 bits, and the only cell outside 2.2σ on the low side anywhere is the control arm’s 6-bit count (61,030 against 61,786, $z = -3.0$, a 1.2% shortfall absent at 8 bits and beyond). The all-ones arm runs high between 18 and 20 bits (at 20 bits, 11 observed against 3.8 expected, $z = +3.7$; conditional halving over steps 16–21 $z = +3.3$) and returns to expectation at 22 (1 against 0.9); the statistic was examined after each of six rounds with a widening window, so we read the surplus as noise and the test as excluding only an obstruction, which is what it was built for.

Honestly stated, the data pin the exact-zero rate only to within a factor of about three: the deepest cell with usable counts is 28 bits (at 24 bits the H100 cell holds 174 candidates against 176.5 expected, which is the depth to which the tests have power), and $P(c_3 = 0) = 2^{-32}$ is an extrapolation over the last four bits. The observed hits in Section 6.3 are consistent with it.

Why we find no steering. For a fixed candidate $(a_0, \dots, a_3)$ the digest and the context words $a_5, \dots, a_{11}$ enter (7) only through the additive constant κ_3 ; the word $v = a_4$ enters W_4 additively before σ_0 , and the context constants also determine which (a_1, a_2, a_3) the three lookups return. Writing $S = \sigma_0(W_4) + W_3$, a sum modulo 2^{32} , the constant contributes only a phase, $\varphi_{c_3}(\beta) = \varphi_S(\beta) e^{-2\pi i \beta \kappa_3 / 2^{32}}$, and if W_3 and $\sigma_0(W_4)$ were independent φ_S would factorise as $\varphi_{W_3} \varphi_{\sigma_0(W_4)}$; the two share a_1, a_2, a_3 , so we treat the factorisation as a model and test it on 40 contexts of about 1,700 candidates each. The largest-root table policy gives W_3 a strong bias ($|\varphi_{W_3}(1)| = 0.155$ per context, a 255σ departure from uniform on the statistic summed over the 40 contexts), and it is annihilated because $|\varphi_{\sigma_0(W_4)}(1)| = 0.025$ per context sits at the noise floor $1/\sqrt{n} = 0.024$; under the model the product, 0.004, lies below that floor, and the observed $|\varphi_{c_3}(1)| = 0.026$ is at the floor, i.e. noise. Under that model, and at our sample size, no table representative policy, even one driving $|\varphi_{W_3}|$ to 1, would move $|\varphi_{c_3}(1)|$ above 0.025, worth at most 9×10^{-4} bits; whether a policy could induce dependence between W_3 and $\sigma_0(W_4)$ is excluded only by the direct tests below; and “some contexts are cheaper” is identically the statement “ S is non-uniform”, for which an 8.4×10^6 -frequency Fourier scan finds no evidence (largest statistic 30.4 against an expected maximum of 31.9 under uniformity, no exceedances), while the same scan on the known-biased W_3 returns 3,235, so the test has power. A policy that

Table 3: The 20-round campaign. Every pod built three root tables on the device. “Expected” is $\text{candidates} \times 2^{-32}$.

GPU	kernel	target	contexts	swept a_0	expected	outcome
H100	PyTorch	random digest	7,351	3.16×10^{13}	0.690	none
A100	PyTorch	random digest	1,839	7.90×10^{12}	0.173	none
A100	PyTorch	random digest	1,836	7.89×10^{12}	0.172	none
A100	Triton	random digest	1,765	7.58×10^{12}	0.166	preimage , 2.45 h
A100	Triton	all-ones	12,203	5.24×10^{13}	1.145	preimage , 16.16 h
A100	Triton	all-ones	12,075	5.19×10^{13}	1.133	none (stopped)
total				1.59×10^{14}	3.48	2 preimages

stores roots on a density- 2^{-k} subset would gain k bits of structure in W_3 and lose exactly 2^{-k} in lookup hit rate. Mutual information between c_3 and every quantity available before the lookups, a_0 , v , the context index and their bit fields, is at the shuffle-null floor (maximum 1.6×10^{-4} bits over 15,360 tests, against a null 99.99th percentile of 1.59×10^{-4}); 17.3×10^6 random linear approximations from a_0 to c_3 give a best correlation of 5.49σ against 5.77σ expected for pure noise.

6.2 Cost

Per swept a_0 , the probability of a 20-round preimage is $0.9337 \cdot (3/4)^{32} \cdot 2^{-32} = 2.18 \times 10^{-14}$ with three-root tables, i.e. $2^{45.4}$ swept a_0 per preimage ($2^{47.3}$ with one root). At 8.5×10^8 per second on an A100 that is a mean of about 15 h, about \$24 at \$1.59 per hour; on an H100, where we did not run the fused kernel, scaling by the eager kernel’s measured ratio (4.47 against 2.54×10^8 swept a_0 per second) would give about 8 h at a higher hourly rate. Being a memoryless search, the time to the first preimage is exponentially distributed: 63% within one mean, 95% within three.

6.3 The computed preimages

Table 3 lists every GPU run (omitting about 48 contexts, 2×10^{11} swept a_0 , on random targets from the two all-ones pods before they were retargeted; `FLEET.txt`). Two preimages were found from a total expectation of 3.48; the probability of at most two is 0.32.

Preimage of a solver-generated digest. The target is the 20-round digest of a random 55-byte message drawn by the solver, so a preimage was known to exist; the block found is a different one.

```
target: d962ca30 635f9b74 ac6c8c12 43a1a9cf 800e81bc 05f1d2e4 0764c68c 795f7388
block:  a36f4238 f2c9204e b5b6653b 070401f7 5928e4f3 fe766be2 52026907 f7ee1812
       01344603 ea505012 86cbe6cb 6ce73d0b 5c91de6a 43355e3b ff3d5e88 c1ad0b54
```

Found 2026-09-04 in context 1,765 after 2.45 h on one A100 (7.58×10^{12} swept a_0). The recovered state has $a_4 = a_5 = 0x799f3424$, $e_8 = e_9 = 0xFFFFFFFF$ and satisfies (6).

Preimage of the all-ones digest. Target `ff...ff` (256 one-bits), the digest Zaikin [12] inverted at 19 rounds and chose because, for an irregular output, “it may be hard to justify this choice and prove that a random pair of input and output was not picked” ([12], §5.1); no 20-round preimage of it was known to exist:

```
0b0187bf b9aea692 b66effa5 087dca3a 0caea827 1d2f9916 739a224e a87c0eae
7f9ef4a7 b318a7de a8848c61 a7a141a4 11ac114b 952299be 97aaa67f c6acfe57
```

Found 2026-09-05 in context 12,203 after 16.16 h on one A100 (5.24×10^{13} swept a_0 ; seed 3971886063), about \$26; a second A100 on the same digest (seed 1138247750) swept 5.19×10^{13} in 12,075 contexts without a hit before being stopped. The recovered state has $a_4 = a_5 = 0x19eb97f7$, $e_8 = e_9 = 0xFFFFFFFF$ and satisfies (6).

Both blocks verify with `code/verify_r19.py --rounds 20`, fail with `--rounds 19` and `--rounds 21`, and reproduce the digest under an independent from-scratch SHA-256. Appendix A gives the commands.

The same digest, one more round. Zaikin inverted the all-ones digest at 19 rounds in 18 h 33 min on 192 CPU cores and stated that inverting the 20-round compression function would likely require additional effective algorithmic techniques. The same digest is inverted here at 20 rounds in 16 h on one GPU.

6.4 Validation of the pipeline

Because no computed 20-round preimage was available to us before the campaign, the fourth-constraint code path had never been exercised on a genuine input, and an error in it would leave the residual looking uniform while silently discarding real hits. We therefore *planted* 20-round preimages: blocks constructed so that their internal state lies in the family and satisfies (6), then hashed for 20 rounds. Run over a window containing the true a_0 , the reference implementation found every plant whose table roots were stored (10 of 16 with two tables; an independent replication found 54 of 200 with one table and 145 of 200 with two, against the expected recovery fractions $c^3 = 0.25$ and $0.8979^3 = 0.72$) and never a wrong one. The Triton kernel was then checked against the reference on 12 plants with two and three tables, agreeing on every hit, and an interpreter-mode execution compared all 27 root-combination paths against the reference on identical inputs.

7 The Barrier at Twenty-One Rounds

Each round beyond 19 adds one schedule constraint. The companion note [2] showed that, for random contexts, the fourth constraint cannot absorb a fifth unknown because the resulting dependency graph contains a cycle whose heavy edge is $a_4 \rightarrow C_0$ through $\Sigma_0(a_4)$ in W_9 . The submask family does not cancel that edge; it removes a different dependency ($a_3 \rightarrow C_1$) and pays C_3 in full. The question is whether any context choice can go further.

Which dependencies a context can clear. The mechanism of Proposition 1 clears a dependency only when it passes through Maj or Ch with a context word among the arguments, because a context word can saturate those functions. A dependency that passes through Σ_0 or σ_0 of an unknown contains no context word and cannot be cleared. For C_3 to absorb a_4 one would need C_1 free of a_3 and of a_4 , and C_2 free of a_4 . Table 4 reports, for six condition families, which of these hold, measured by perturbing each unknown and observing the constraint targets.

The middle column never turns. With $a_5 = a_4$ the a_4 -dependence of C_1 is $\Sigma_0(a_4)$ inside $T_1^{(6)}$; without it, a_3 leaks back in through $\text{Maj}(a_5, a_4, a_3)$. Either way a non-maskable path remains. A symbolic search over 1,485 legal context-condition families (`analysis/scripts/symdep/`) confirmed that none clears it. The collapse of Proposition 2 survives the tighter families (collapse rates 1.016×10^{-4} and 0.987×10^{-4} , `logs/frame_search.log`), so they cost nothing but also gain nothing.

Independence of the fourth and fifth constraints. On 214,583 freshly generated 21-round candidates (84 targets, 8.4×10^9 swept a_0 ; `analysis/scripts/symdep/gen.log`) satisfying

Table 4: Which above-diagonal dependencies each context family clears at 20 rounds (measured on 40 random instances each by `scripts/frame_search2.py`; output in `logs/frame_search.log`).

Context conditions	C_1 free of a_3	C_1 free of a_4	C_2 free of a_4
random	no	no	no
$a_4 = a_5, e_8 = e_9 = -1$ (this family)	yes	no	no
... and $a_6 = a_5$	yes	no	no
$a_5 = a_6, e_9 = e_{10} = -1$ (shifted)	no	no	yes
$a_4 = a_5 = a_6, e_8 = e_9 = e_{10} = -1$	yes	no	yes
$a_4 = \dots = a_7, e_8 = \dots = e_{11} = -1$	yes	no	yes

everything up to the collapsed condition, the residuals of C_3 (at W_{19}) and C_4 (at W_{20}) are each uniform and their mutual information is below 10^{-5} bits. On this evidence we model 21 rounds as costing this construction the full 2^{64} above nineteen, about 2^{77} swept a_0 with three-root tables (2^{79} with one), beyond reach; the measurement bounds the dependence between the two filters, it does not prove exact independence.

What the preimage set does not contain. We mined 51,268 verified 19-round preimages and 68,680 20-round candidates for structure not forced by the construction: per-bit biases, GF(2)-linear and affine relations, context effects, neighbour structure in a_0 , residue-class structure in c_3 , and the entropy accounting of Proposition 2. The 32 per-bit-position entropies of (a_2, a_3) sum to 50.7189 bits against the predicted $32 \log_2 3 = 50.7188$ (each position 1.58496 ± 0.00003), and the 496 pairwise mutual informations between positions sum to 0.001 bits, so the surviving pairs are consistent with uniformity on the allowed set at first and second order; higher-order dependence is not excluded by this test, and its practical effect is bounded only by the yield audit and the Fourier scan above; the GF(2) rank deficit is exactly the three carry-free least-significant bit relations the construction implies; and no other test departed from its null. Seven separate automated analyses (see the disclosure) proposed eight candidate levers, and a separate refutation pass rejected every one on re-examination. One methodological note: a corpus that draws a_0 with replacement contains duplicate rows that produce a spurious 69σ “collision” signal in c_3 until deduplicated.

8 Prior Work and Priority

Computed preimages of round-reduced SHA-256. 16 rounds by SAT in 2012 [8]; 17 and 18 rounds for the all-zero digest, 19 only in a weakened form (at most 31 of the 256 digest bits fixed, to zero: 32 instances at one-bit steps, all but the 32-bit one solved), and 19-round collisions, by Davydov et al. [9]; 19 rounds for the all-ones digest by Zaikin [12], extending [10, 11]; and 19 rounds for arbitrary digests, seven preimages, by our earlier paper [1]. Before submitting we searched the IACR ePrint archive and proceedings through September 2026, dblp, OpenAlex, arXiv, the main SAT, CP and AI venues where the practical line lives (the SAT 2026 and CP 2025 proceedings in full), Zaikin’s repositories, and Russian- and Chinese-language sources, from four independent search passes. No computed preimage of the SHA-256 compression function at 20 or more rounds was found, in any model, weakened or not. We did not enumerate the CP 2026, IJCAI 2026 or AAAI 2026 proceedings, doctoral theses, or SAT-competition solver logs; the claim is scoped accordingly.

Theoretical preimage attacks. The meet-in-the-middle line [4, 5, 6] reaches 45 steps at $2^{255.5}$; Guo et al. [7] extend it to 46 steps at $2^{254.5}$ and 47 at $2^{255.6}$ (pseudo-preimages at $2^{253.2}$), improve 43–45 steps, take SHA-512 to 51 steps, and describe the preceding decade as stagnant.

None of these attacks has been executed; their preimages are obtained from pseudo-preimages by the generic conversion and pay for padding. Rounds and steps coincide for SHA-256 in both literatures, so no prior result becomes twenty under a different convention. The citation runs one way: the SAT line quotes the meet-in-the-middle bounds [12, 9], but [7] contains no SAT-based preimage work.

Computed partial results that must not be conflated. The artefact accompanying [7] (the repository `github.com/desert-oasis/dual-sync-MITM`, directory `SHA256-44_partial_pseudo_preimage`) computes messages for 44-step SHA-256 whose output has 32 or 40 of 256 bits fixed to zero, from a free chaining value: weakened on three axes at once. Bouam et al. [13] exhibit three messages whose *full* SHA-256 digests XOR to zero on their first 128 bits (a 3XOR), a brute-force computation of at least 8.7×10^{22} integer operations by the authors’ count; that is a collision-type result on half the output, not a preimage of any bits. The GraphBench benchmark [16], building on the encodings of [14, 15], emits instances labelled as SHA-256 preimages at 16–30 rounds, but with up to 384 of the 512 input bits fixed, and no round count has been claimed from them.

The levers. Choice-function absorption is the message-stealing tool of [5]; majority degeneracy is formalised in [7]; our companion note [2] records both on a different constraint. The pairing of two context words, the application to the iteration-carrying constraint, and the bitwise collapse of the modular consistency residual are, to our knowledge, new.

9 Relation to Our Earlier Papers

This paper supersedes the cost figures of [1] and the headline of [2], and we say so precisely rather than silently.

- [1] reports about one preimage per 80 attempts of 2^{32} swept a_0 , i.e. about $2^{38.3}$ swept a_0 , and about two GPU-hours per 19-round preimage with random contexts. Those figures remain correct for that attack; the submask family is a different attack with the same tables and backward chain, at $2^{15.26}$.
- [2] bounds the family at about 2^{64} per context-equivalent at 20 rounds and locates the obstruction in $\Sigma_0(a_4)$ inside W_9 . The factor 2^{32} for the fourth constraint stands, and the location of the obstruction stands (Section 7); what changes is the per-context cost of the first three constraints, which the family reduces from a lossy iteration to three lookups. The note’s own caveat, that its negative results are properties of the parameterisations tried, is exactly what applied: its section on a context family that removes a_2 from C_0 applies the same two levers to C_0 and records the result as a structural observation rather than an attack step, because the Maj condition it chose tied a context word to an unknown; applying them to C_1 with a pairing on two context words is the step it did not take.
- The screening campaign of [1] reports a $3.41\times$ lift in a downstream counter. That figure is trajectory-counted; duplicates are 0.03% of converged trajectories, and identity-level deduplication can move it only within $[2.42\times, 3.44\times]$. It is now of historical interest only.

10 Conclusion

Imposing three conditions on the context words rather than drawing them all at random removes the iteration from the 19-round attack and collapses its final check to a bitwise condition, reducing the cost of a 19-round preimage of the SHA-256 compression function by about 2^{23} ,

to milliseconds on one CPU core, with the measured rate within one per cent of the derived one. The same construction, with a fused GPU kernel and three-root tables, computes 20-round preimages at about $2^{45.4}$ swept a_0 , roughly fifteen hours on one A100; two are exhibited, one for the all-ones digest previously inverted at 19 rounds on 192 cores. To our knowledge, as of September 2026 and after the search described in Section 8, no computed preimage of the SHA-256 compression function beyond 19 rounds had previously been published. For this construction, twenty-one rounds costs the full additional 2^{64} on the evidence we have, for a reason we can state exactly: the surviving dependency passes through Σ_0 of the unknown, which no context word can saturate. We do not claim that this is fundamental. Full SHA-256 is untouched.

Acknowledgements and disclosure

The derivations, code, experiments, verification and text of this paper were developed with substantial assistance from an AI system (Anthropic’s Claude, used through Claude Code) operating under the author’s direction, including the design and execution of the GPU campaign and the independent verification passes described in Appendix B. All results were verified by code that shares nothing with the attack implementation and can be reproduced from the repository. The independent replications, reviews, corpus analyses and literature searches cited in Sections 5–8 and in Table 1 were carried out by separate AI-agent sessions working from fresh implementations without access to the solver code, not by human referees; their full reports are in `experiments/submask_2026-09-03/reviewers/`, `analysis/` and `priority/`.

References

- [1] K. S. Basra. A practical preimage attack on the 19-round SHA-256 compression function via a global σ_0 -difference table. Manuscript, 2026. Source, code and witnesses at <https://github.com/kamb-code/sha256-r19-preimage>.
- [2] K. S. Basra. The 20-round barrier for table-based preimage attacks on SHA-256. Companion note, 2026. Source and code at <https://github.com/kamb-code/sha256-r19-preimage>.
- [3] National Institute of Standards and Technology. *FIPS PUB 180-4: Secure Hash Standard (SHS)*, 2015.
- [4] T. Isobe and K. Shibutani. Preimage attacks on reduced Tiger and SHA-2. In *Fast Software Encryption (FSE 2009)*, LNCS 5665, pp. 139–155. Springer, 2009.
- [5] K. Aoki, J. Guo, K. Matusiewicz, Y. Sasaki, and L. Wang. Preimages for step-reduced SHA-2. In *Advances in Cryptology – ASIACRYPT 2009*, LNCS 5912, pp. 578–597. Springer, 2009.
- [6] D. Khovratovich, C. Rechberger, and A. Savelieva. Bicliques for preimages: attacks on Skein-512 and the SHA-2 family. In *Fast Software Encryption (FSE 2012)*, LNCS 7549, pp. 244–263. Springer, 2012.
- [7] J. Guo, H. Li, M. Liu, S. Wang, and T. Zhang. Dual-syncopation meet-in-the-middle attacks: new results on SHA-2 and MD5. In *Advances in Cryptology – EUROCRYPT 2026*, LNCS 16546, pp. 121–151. Springer, 2026. DOI 10.1007/978-3-032-25333-0_5. Cryptology ePrint Archive, Report 2026/353.

- [8] E. Homsirikamol, P. Morawiecki, M. Rogawski, and M. Srebrny. Security margin evaluation of SHA-3 contest finalists through SAT-based attacks. In *Computer Information Systems and Industrial Management (CISIM 2012)*, LNCS 7564, pp. 56–67. Springer, 2012. DOI 10.1007/978-3-642-33260-9_4.
- [9] V. V. Davydov, M. D. Pikhtovnikov, A. P. Kiryanova, and O. S. Zaikin. Analysis of the cryptographic strength of the SHA-256 hash function using the SAT approach. *Scientific and Technical Journal of Information Technologies, Mechanics and Optics*, 25(3):428–437, 2025. DOI 10.17586/2226-1494-2025-25-3-428-437.
- [10] O. Zaikin. Inverting step-reduced SHA-1 and MD5 by parameterized SAT solvers. In *30th International Conference on Principles and Practice of Constraint Programming (CP 2024)*, LIPIcs 307, pp. 31:1–31:19, 2024. DOI 10.4230/LIPIcs.CP.2024.31.
- [11] O. Zaikin. Inverting cryptographic hash functions via Cube-and-Conquer. *Journal of Artificial Intelligence Research*, 81:359–399, 2024. DOI 10.1613/jair.1.15244.
- [12] O. Zaikin. Preimage attacks on round-reduced MD5, SHA-1, and SHA-256 using parameterized SAT solver. *Constraints*, 2026 (received 28 March 2025, accepted 16 December 2025, published online 23 January 2026). DOI 10.1007/s10601-025-09383-0.
- [13] M. Bouam, C. Bouillaguet, C. Delaplace, and C. Noûs. Computational records with aging hardware: controlling half the output of SHA-256. *Parallel Computing*, 106:102804, 2021. DOI 10.1016/j.parco.2021.102804. Cryptology ePrint Archive, Report 2021/886.
- [14] S. Nejati, J. H. Liang, C. Gebotys, K. Czarnecki, and V. Ganesh. Adaptive restart and CEGAR-based solver for inverting cryptographic hash functions. In *Verified Software: Theories, Tools, and Experiments (VSTTE 2017)*, LNCS 10712, pp. 120–131. Springer, 2017. DOI 10.1007/978-3-319-72308-2_8.
- [15] N. Alamgir, S. Nejati, and C. Bright. SHA-256 collision attack with programmatic SAT. In *Joint Proceedings of the 9th Workshop on Practical Aspects of Automated Reasoning (PAAR) and the 9th Satisfiability Checking and Symbolic Computation Workshop (SC-Square)*, CEUR Workshop Proceedings vol. 3717, pp. 91–110, 2024.
- [16] T. Stoll, C. Qian, B. Finkelshtein, A. Parviz, D. Weber, F. Frasca, H. Shavit, A. Siraudin, A. Mielke, M. Anastacio, E. Müller, M. Bechler-Speicher, M. Bronstein, M. Galkin, H. Hoos, M. Niepert, B. Perozzi, J. Tönshoff, and C. Morris. GraphBench: next-generation graph learning benchmarking. arXiv:2512.04475 (v5, May 2026), 2025.

A Witnesses and verification

Both blocks below are preimages of the 20-round SHA-256 compression function with the standard initial value, feed-forward retained and no padding constraint. The verifier is the standard-library script `code/verify_r19.py` from the repository; it shares no code with the attack.

```
# all-ones digest (the target of Zaikin, Constraints 2026)
python3 code/verify_r19.py --rounds 20 \
  --hash ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff \
  --words "0b0187bf b9aea692 b66effa5 087dca3a 0caea827 1d2f9916 739a224e a87c0eae
          7f9ef4a7 b318a7de a8848c61 a7a141a4 11ac114b 952299be 97aaa67f c6acfe57"

# solver-generated digest
python3 code/verify_r19.py --rounds 20 \
  --hash d962ca30635f9b74ac6c8c1243a1a9cf800e81bc05f1d2e40764c68c795f7388 \
  --words "a36f4238 f2c9204e b5b6653b 070401f7 5928e4f3 fe766be2 52026907 f7ee1812
          01344603 ea505012 86cbe6cb 6ce73d0b 5c91de6a 43355e3b ff3d5e88 c1ad0b54"
```

Each prints result: OK; with `--rounds 19` or `--rounds 21` each prints result: FAIL.

B Reproducibility

All code, logs, reviewer artefacts and this paper are in the repository at <https://github.com/kamb-code/sha256-r19-preimage>, under `code/` and `experiments/submask.2026-09-03/`.

- `code/submask_family.py`: the family, the triangular solve, the collapsed test, a table-free algebraic self-test of Propositions 1 and 2 (run with no arguments), and `--attack / --measure` modes.
- `code/build_sigma0_table.py`: builds the $\sigma_0(u) - u$ table (int32, sentinel -1 , largest root per value; about 215 core-seconds).
- `code/gpu_submask.py`: the PyTorch sweep with up to three root tables built on the device; `code/submask_triton.py`: the fused kernel with `--verify` (planted preimages) and `--bench`.
- `experiments/submask.2026-09-03/`: as-run scripts and logs with a sha256 manifest; `FLEET.txt` (every pod, GPU, target, seed, exposure and cost); `NOTES.md` (corrections found by review); `R20_PREIMAGE_ALLONES.txt` and `R20_PREIMAGE.txt` (witnesses and provenance); `logs/r19_fixed_targets.log`, `logs/frame_search.log` and `logs/ones_obstruction.log` (the fixed-target runs, the context-family search of Table 4, and the all-ones obstruction test); `analysis/` (the corpus mining); `priority/` (the literature search and adjudication); `reviewers/` (independent verification scripts, including three SHA-256 implementations written from the specification with constants derived from prime roots, and the full review reports).
- Statistical method for nested thresholds: counts n_k of candidates whose residual has its low k bits zero are nested; the ratios n_{k+1}/n_k are independent $\text{Binomial}(n_k, \frac{1}{2})$ proportions and are the statistic to test, with a multiple-comparison correction over the steps examined.